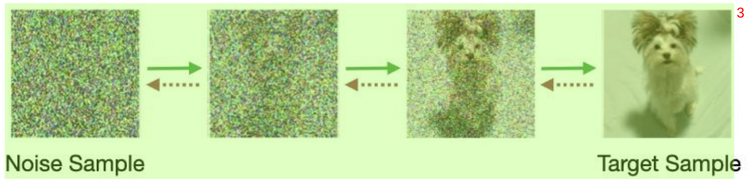
Learning to Reach Goals via Diffusion

Vineet Jain Siamak Ravanbakhsh

Mila McGill

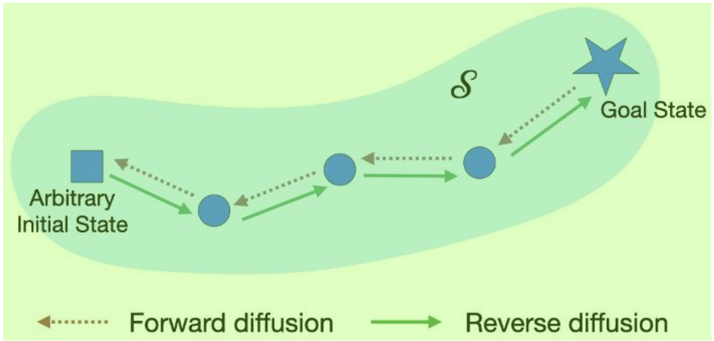
Generate trajectories that "diffuse away" from potential goals, and train a policy to reverse them [1]

Denoising Diffusion Models [2]



[3]

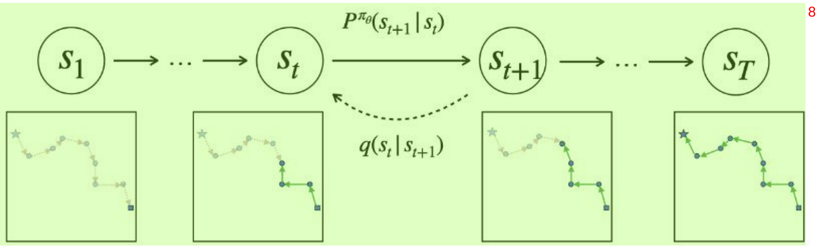
Goal-conditioned Reinforcement Learning [4]



[5]

Goal Conditioned RL as Diffusion [6]

Diffusion models: map any point from Gaussian → data manifold
Goal-conditioned RL: learn optimal path from any state → goal state [7]



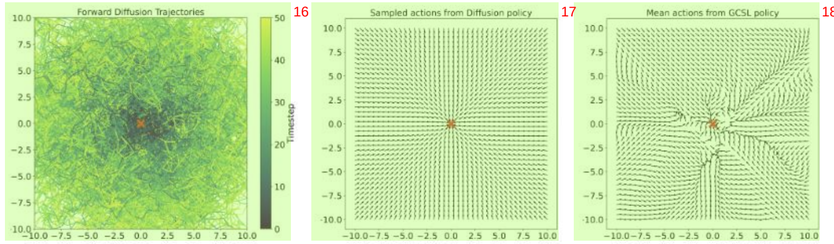
[8]

diffusion model [9]	GCRL (Merlin) [11]
target dataset [10]	goal states [12]
score function	policy
forward process	$q(s_t\|s_{t+1})$
reverse process	$\mathcal{P}^\pi(s_{t+1}\|s_t)$

Actions that lead us away from the goal in place of Gaussian noise; Policy that reverses these actions in place of score function [13]

A Simple 2D Navigation Problem [14]

- **Task:** Navigate to goal X; **States:** (x, y); **Actions:** unit length displacement in (x, y) [15]
- **Forward process:** Take random actions starting from goal
- **Reverse process:** Train policy to reverse the forward trajectories



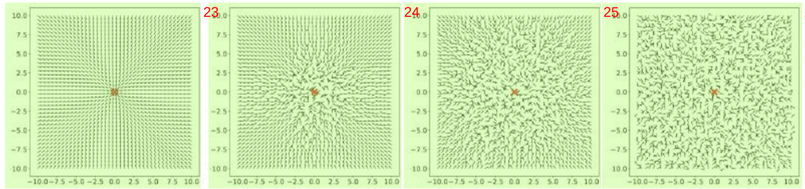
[16] [17] [18]

(a) Visualization of trajectories starting from the goal X generated during the forward process, [19]
(b) Predicted actions from Merlin policy, (c) Predicted actions from GCSL policy

Backward view of diffusion gives us control over the goal distribution [20]

Effect of Time Conditioning [21]

- Analogous to the score function in diffusion, the Merlin policy is conditioned on the time [22]
- This time indicates how "noisy" current state is, or how far we are from the target goal state
- Policy takes optimal action far away from goal and becomes uncertain as it approaches the goal; this uncertainty appears farther from goal as time horizon increases



[23] [24] [25] [26]

(a) h=1 [27] (b) h=5 [28] (c) h=10 [29] (d) h=20 [30]

The time conditioning affects policy uncertainty based on expected number of steps to reach the goal [31]

Optimization Objective [32]

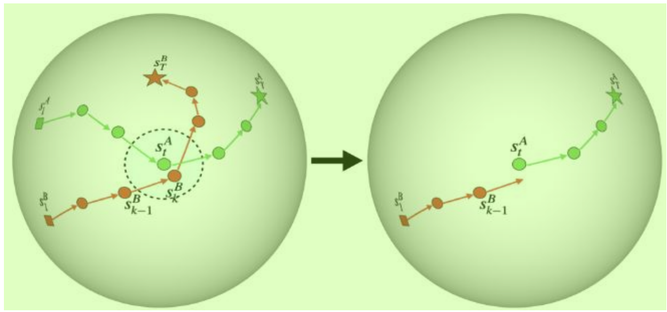
- Given an offline dataset $\mathcal{D}$, we can view the trajectories as forward diffusion chains generated by some unknown policy [33]
- As per below Theorem, behavior cloning maximizes the probability of reaching the goal states

$$\theta^* = \arg\max_\theta \mathbb{E}_{g\sim p(g),(s,a)\sim D(g)}[\log \pi_\theta(a \mid s, g)]$$

Theorem: Given a dataset $\mathcal{D}$ and target goal distribution $p(g)$, behavior cloning using a goal-conditioned policy maximizes a lower bound on the log-likelihood of the goal states $L = \mathbb{E}_{g\sim p(g), s_T\sim q(s_T|g)}[\log p_\theta(s_T)]$. [34]

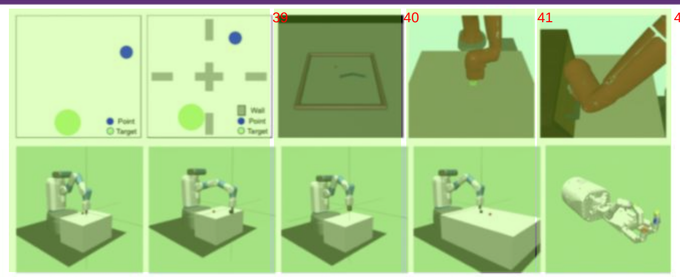
Constructing the Forward Process [35]

- **Merlin:** Use pre-collected offline trajectories, train policy using behavior cloning [36]
- **Merlin-P:** Learn reverse dynamics model to simulate trajectories backward from goal
- **Merlin-NP:** Use proposed trajectory stitching method to construct backward trajectories

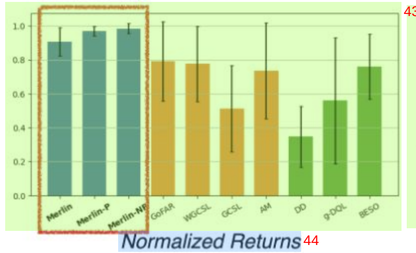


[37]

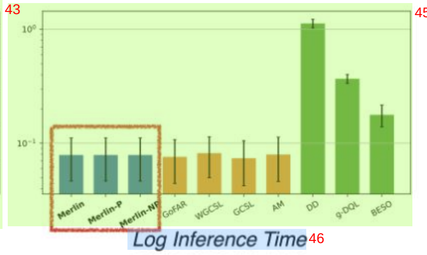
Experimental Results [38]



[39] [40] [41] [42]



Normalized Returns [44]



Log Inference Time [46]

[43] [45]

Merlin outperforms previous methods, while being highly-efficient! [47]

Conclusions and future work [48]

- Merlin constructs trajectories that "diffuse away" from goals and trains a policy to reverse them [49]
- Simple training dynamics indicate potential for scalability, as proven by denoising diffusion models
- Directions for future work include extending to online setting and partially observable environments

paper [50] code [52]

[51] [53]